

Flujo de señales

Prototipo 1 — Modelador de Recintos 3D con Simulación Acústica FEM

Ingeniería acústica / Desarrollo de software

Índice

1. Visión general de la arquitectura	2
2. Flujo 1 — Cambio de geometría	2
3. Flujo 2 — Cálculo modal FEM	4
4. Flujo 3 — Evaluación de la FRF	5
5. Flujo 4 — Visualización del campo acústico 3D	6
6. Flujo 5 — Posicionamiento y arrastre de fuentes	7
7. Flujo 6 — Slice 2D horizontal	8
8. Flujo 7 — RT60 y amortiguamiento desde materiales	9
9. Tabla resumen de señales Qt	10
10. Ciclo de vida del ModalSolution	10
11. Consideraciones de implementación	10
11.1. Orden de renderizado en OpenGL	10
11.2. Proyección manual de la cámara	11
11.3. Debounce del campo acústico	11
11.4. Interpolación baricéntrica en mallas tet	11

1. Visión general de la arquitectura

La aplicación está organizada en tres capas: **interfaz (GUI)**, **cómputo** y **renderizado 3D**. La comunicación entre capas ocurre mediante el sistema de *signals/slots* de Qt5; ningún módulo llama directamente a otro sin pasar por una señal o un método público explícito.

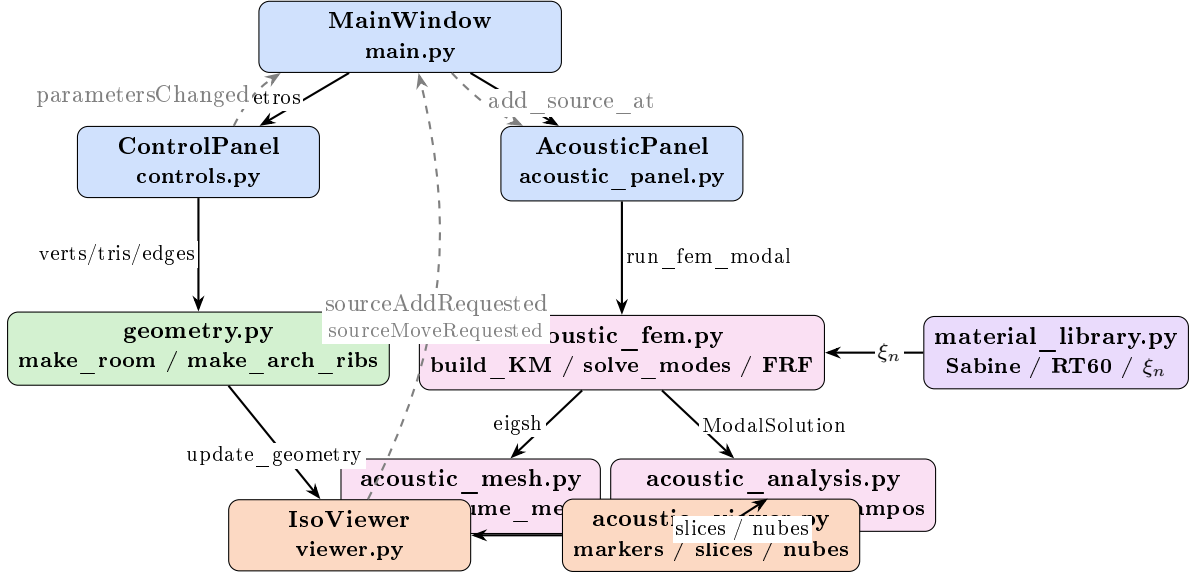


Figura 1: Arquitectura de tres capas. Líneas sólidas = llamadas directas; líneas punteadas = señales Qt.

2. Flujo 1 — Cambio de geometría

Cada vez que el usuario mueve un slider o edita el polígono de planta, **ControlPanel** emite `parametersChanged`. El flujo completo hasta el renderizado se describe a continuación.

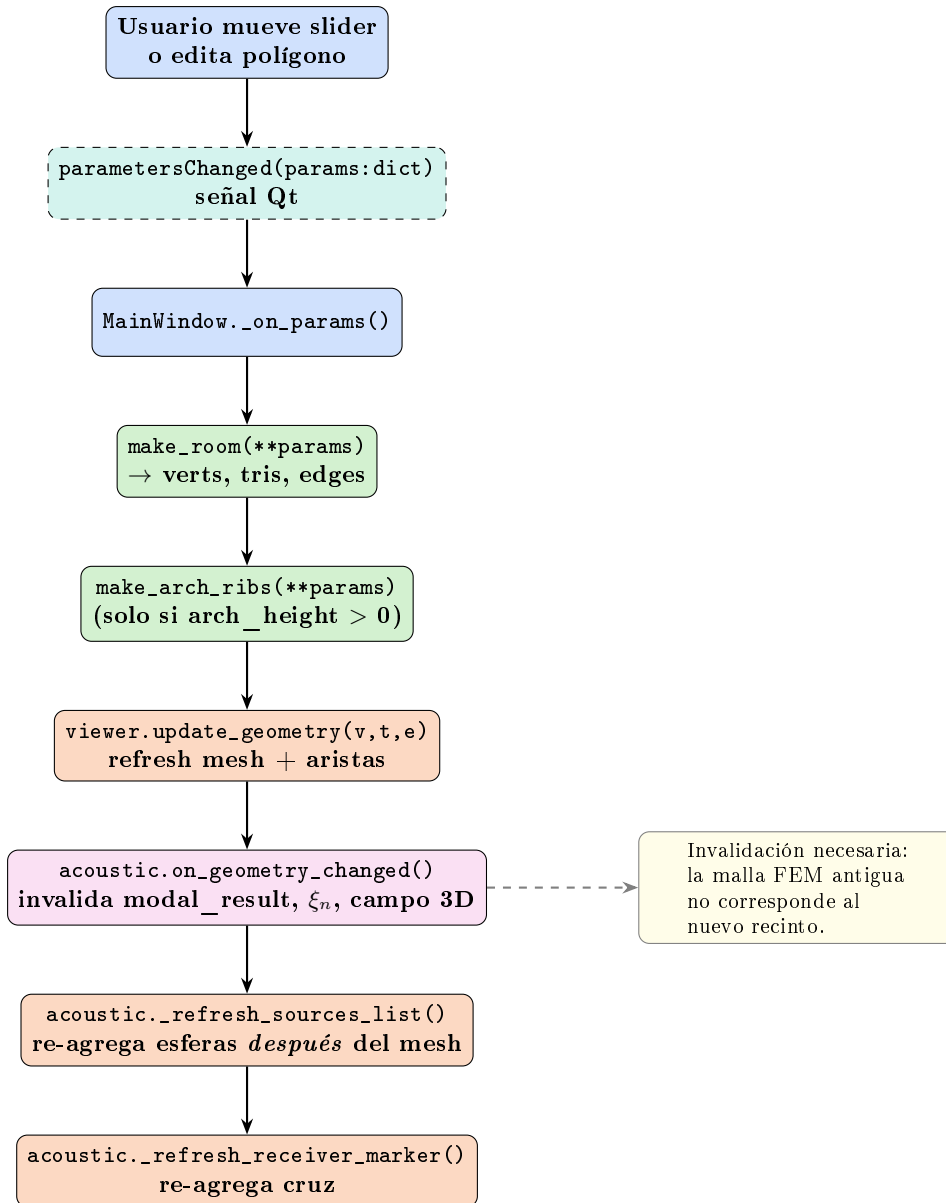


Figura 2: Flujo de cambio de geometría.

Detalles del mallado volumétrico

`build_volume_mesh` construye una malla tetraédrica interior:

1. Calcular AABB (bounding box) del recinto.
2. Generar rejilla estructurada de hexaedros con paso $h = 1/\text{n_per_meter}$.
3. Subdividir cada hexaedro en 6 tetraedros (split de Freudenthal).
4. Filtrar tetraedros cuyo centroide cae *fuera* del recinto (test point-in-polyhedron por rayo en $+z$, paridad de intersecciones).
5. Re-indexar nodos usados.

3. Flujo 2 — Cálculo modal FEM

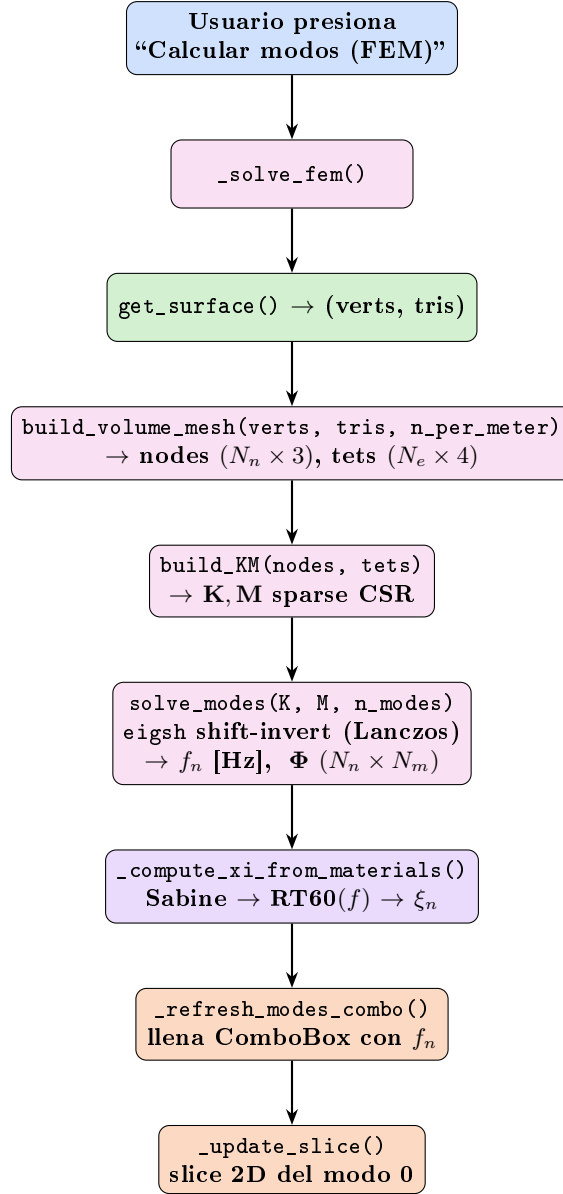


Figura 3: Flujo del cálculo modal FEM.

Formulación variacional

El problema de autovalores acústico con condición de Neumann homogénea (paredes rígidas):

$$\mathbf{K} \phi_n = \lambda_n \mathbf{M} \phi_n, \quad f_n = \frac{c}{2\pi} \sqrt{\lambda_n}$$

donde

$$K_{ij} = \int_{\Omega} \nabla N_i \cdot \nabla N_j \, dV, \quad M_{ij} = \int_{\Omega} N_i N_j \, dV$$

Los modos se M-ortonormalizan: $\phi_n^T \mathbf{M} \phi_m = \delta_{nm}$.

Amortiguamiento modal desde materiales

$$A(f) = \sum_i \alpha_i(f) S_i \quad (\text{absorción de Sabine})$$

$$\text{RT}_{60}(f) = \frac{0,161 V}{A(f)} \quad (\text{Sabine})$$

$$\xi_n = \frac{1,1}{f_n \cdot \text{RT}_{60}(f_n)} \quad (\text{amortiguamiento modal del modo } n)$$

Los coeficientes $\alpha_i(f)$ se obtienen de archivos JSON en la carpeta **materials/** e interpolan en escala logarítmica entre bandas de octava.

4. Flujo 3 — Evaluación de la FRF

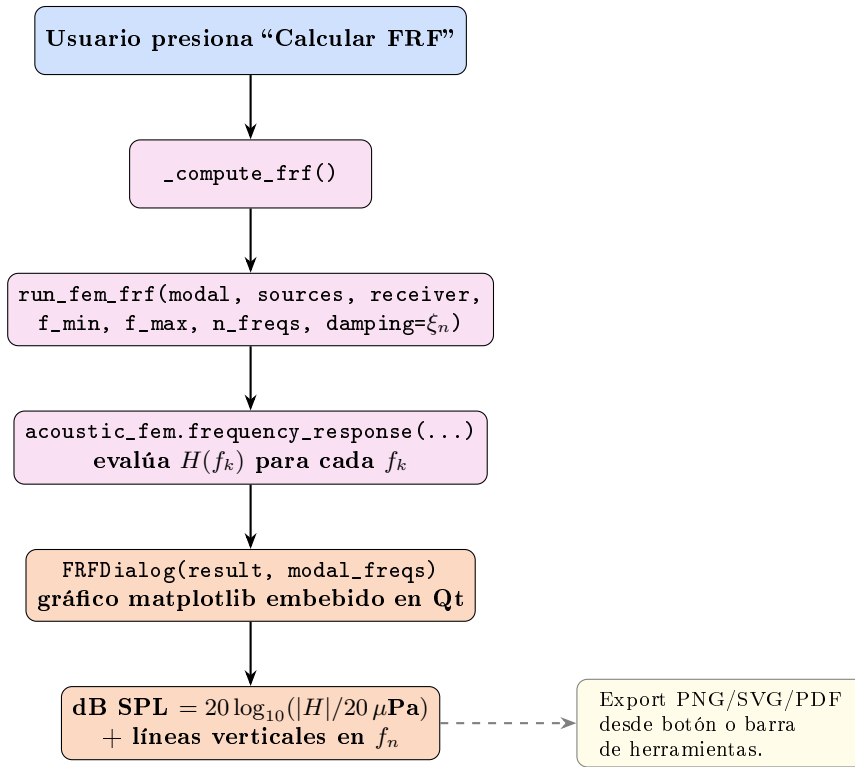


Figura 4: Flujo de cálculo y visualización de la FRF.

Superposición modal

$$H(f) = i\omega\rho_0 \sum_{n=1}^{N_m} \frac{\varphi_n(\mathbf{x}_r) \sum_s Q_s \varphi_n(\mathbf{x}_s)}{\omega_n^2 - \omega^2 + 2i\xi_n \omega_n \omega}$$

con $\omega = 2\pi f$, $\rho_0 = 1,21 \text{ kg/m}^3$. El denominador se evalúa en cada frecuencia f_k del eje de la FRF; el numerador (independiente de ω) se pre-calcula.

5. Flujo 4 — Visualización del campo acústico 3D

El botón “Actualizar campo 3D” (o la tecla Enter) despacha `_update_field_3d()`, cuyo comportamiento depende del `combo_field`:

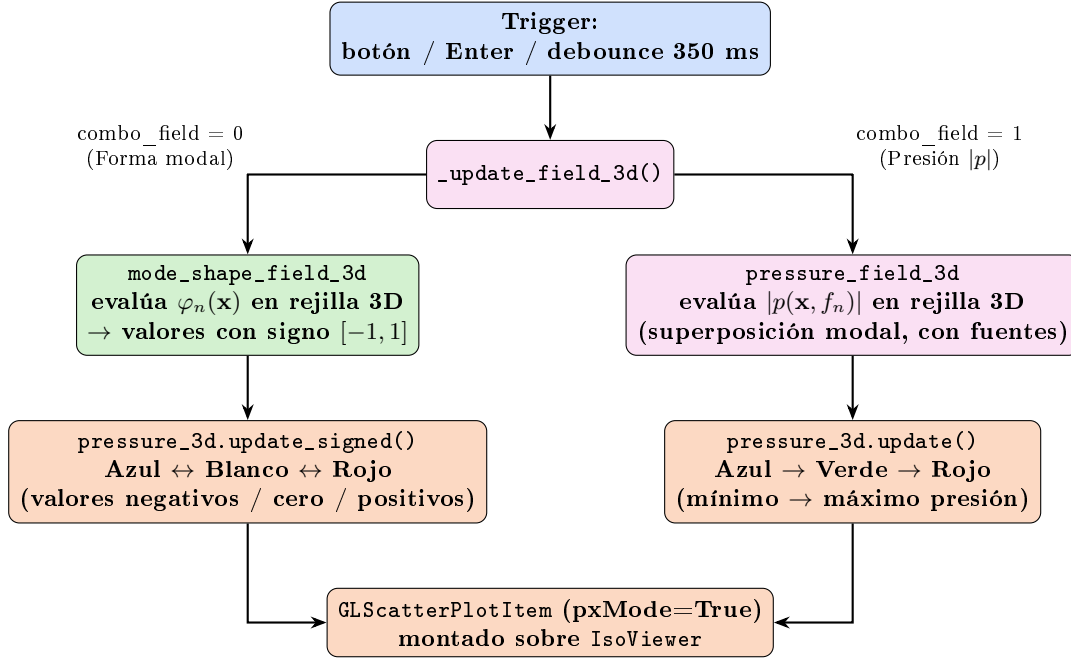


Figura 5: Bifurcación del flujo de visualización según `combo_field`.

Rejilla de evaluación

Los campos se evalúan sobre una rejilla uniforme 3D dentro del AABB de la malla FEM. Solo se conservan los puntos donde el interpolador baricéntrico devuelve un valor finito (puntos dentro de algún tetraedro):

$$\{(x_i, y_i, z_i)\}_{i=1}^N = \{\mathbf{x} \in \mathbb{R}^3 \mid \text{FEM-locator}(\mathbf{x}) \neq \text{NaN}\}$$

Por qué forma modal y presión se parecen *en resonancia*

En la frecuencia $f = f_n$ (resonancia del modo n):

$$p(\mathbf{x}) \approx \frac{\rho_0}{2\xi_n \omega_n} \cdot \varphi_n(\mathbf{x}_s) \cdot \varphi_n(\mathbf{x})$$

La distribución espacial de $|p(\mathbf{x})|$ es proporcional a $|\varphi_n(\mathbf{x})|$. La diferencia observable es el *signo*: la forma modal muestra lóbulos + y - (azul/rojo), mientras que $|p|$ es siempre no-negativo (solo rojo).

Al mover la fuente a un **nodo** del modo n ($\varphi_n(\mathbf{x}_s) = 0$), la contribución del modo n se anula y el patrón cambia significativamente (dominan otros modos fuera de resonancia).

6. Flujo 5 — Posicionamiento y arrastre de fuentes

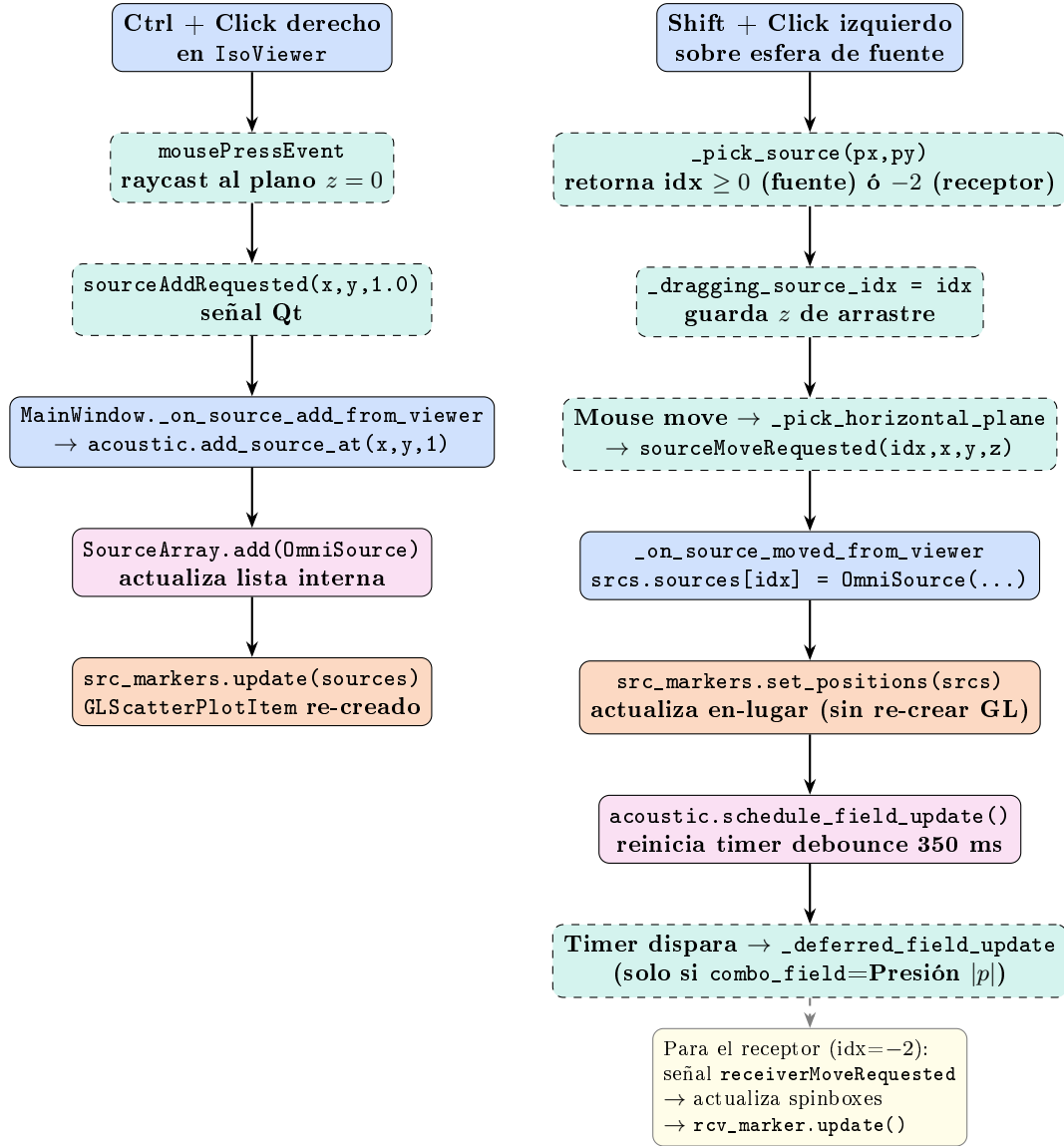


Figura 6: Flujo de posicionamiento (izquierda) y arrastre (derecha) de fuentes acústicas.

Raycast para posicionamiento

El click en el visor 3D lanza un rayo desde el ojo de la cámara en la dirección del pixel seleccionado. La intersección con el plano $z = z_0$ se calcula como:

$$t = \frac{z_0 - o_z}{d_z}, \quad \mathbf{x} = \mathbf{o} + t \mathbf{d}$$

donde $\mathbf{o}$ es el origen del rayo y $\mathbf{d}$ su dirección, obtenidos invirtiendo la matriz de proyección-vista construida manualmente con `QMatrix4x4`.

Detección de fuentes en pantalla (*picking*)

Para Shift+Click, la función `_pick_source(px,py)` proyecta cada posición de fuente (x_s, y_s, z_s) al espacio de pantalla y busca la más cercana dentro de un radio de 28 px:

$$i^* = \arg \min_i [(px - u_i)^2 + (py - v_i)^2], \quad \text{si } < 28^2 \text{ px}^2$$

donde (u_i, v_i) es la proyección de la fuente i . El receptor usa el índice especial -2 .

7. Flujo 6 — Slice 2D horizontal

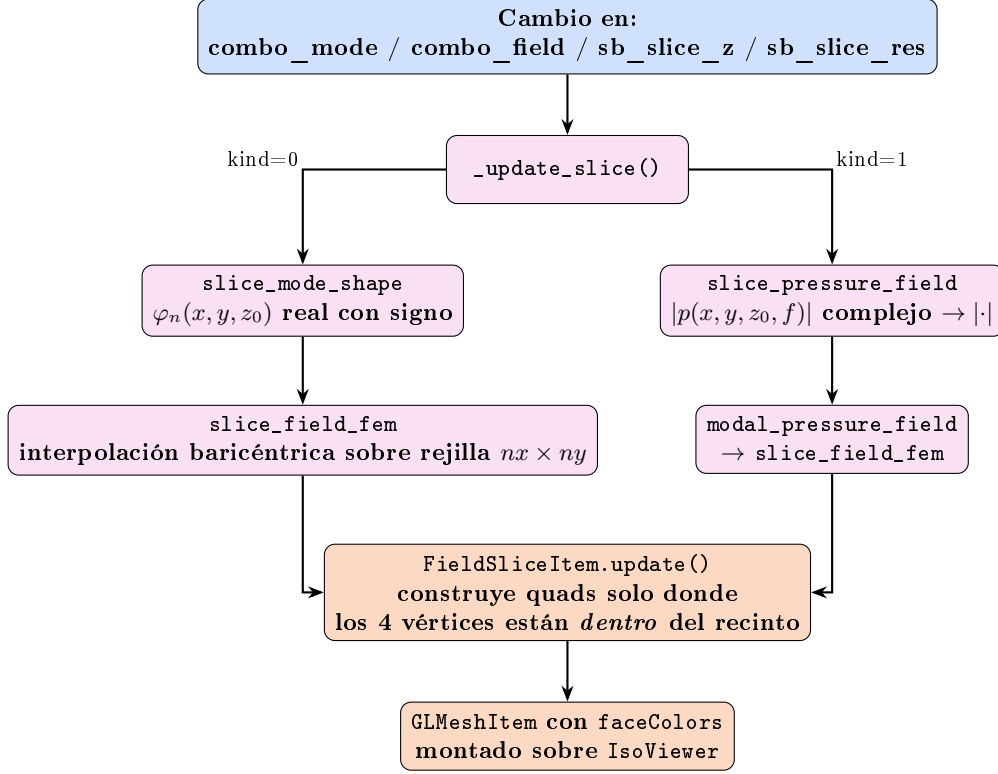


Figura 7: Flujo del slice 2D horizontal.

Enmascaramiento dentro del recinto. Un quad de la rejilla se incluye solo si sus cuatro vértices tienen coordenadas dentro del mesh FEM (el interpolador devuelve un valor finito):

$$\text{mostrar quad}_{ij} \iff \text{mask}[i, j] \wedge \text{mask}[i+1, j] \wedge \text{mask}[i, j+1] \wedge \text{mask}[i+1, j+1]$$

Esto evita que el slice “sangre” fuera del contorno del recinto.

8. Flujo 7 — RT60 y amortiguamiento desde materiales

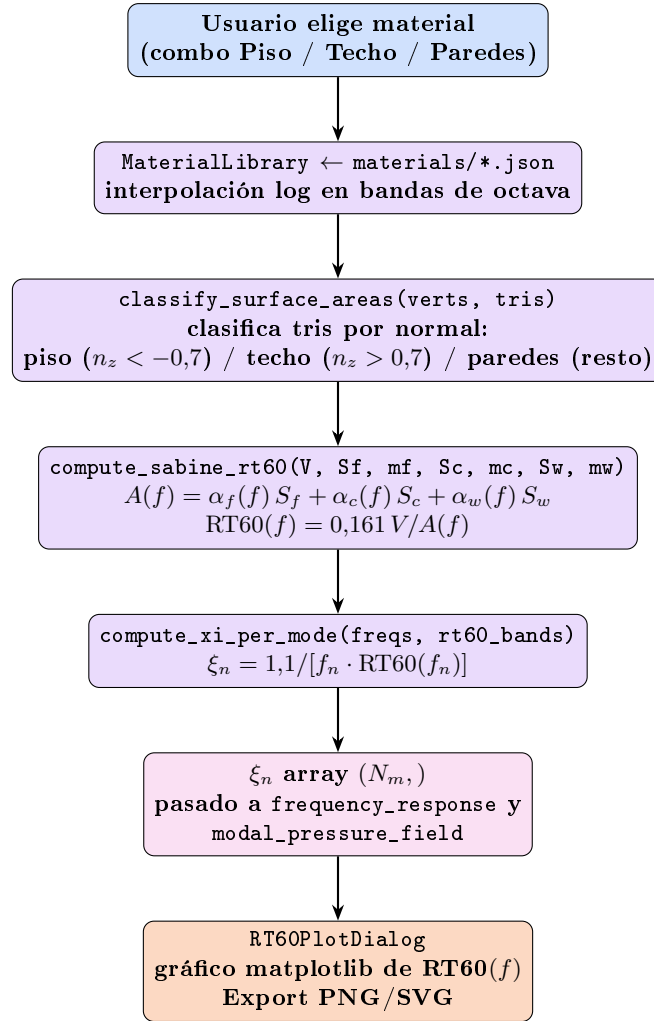


Figura 8: Flujo de cálculo de amortiguamiento modal desde materiales.

9. Tabla resumen de señales Qt

Señal	Emisor	Slot receptor
parametersChanged(dict)	ControlPanel	MainWindow._on_params
parametersCommitted(dict)	ControlPanel	MainWindow._on_commit
cameraResetRequested()	ControlPanel	IsoViewer.reset_camera
drawShapeRequested()	ControlPanel	MainWindow._open_shape_dialog
showLabelsToggled(bool)	ControlPanel	IsoViewer.set_show_labels
viewModeChanged(str)	ControlPanel	IsoViewer.set_view_mode
wallDragStarted(int)	IsoViewer	ControlPanel.begin_wall_drag
wallDragMoved(float)	IsoViewer	ControlPanel.update_wall_drag
wallDragEnded()	IsoViewer	ControlPanel.end_wall_drag
sourceAddRequested(x,y,z)	IsoViewer	MainWindow._on_source_add
sourceEditRequested(int)	IsoViewer	MainWindow._on_source_edit
sourceMoveRequested(i,x,y,z)	IsoViewer	MainWindow._on_source_moved
receiverMoveRequested(x,y,z)	IsoViewer	MainWindow._on_receiver_moved
sourceAddedAtFloor(x,y)	ShapeCanvas	MainWindow._on_source_add

Cuadro 1: Tabla de señales Qt en la aplicación.

10. Ciclo de vida del ModalSolution

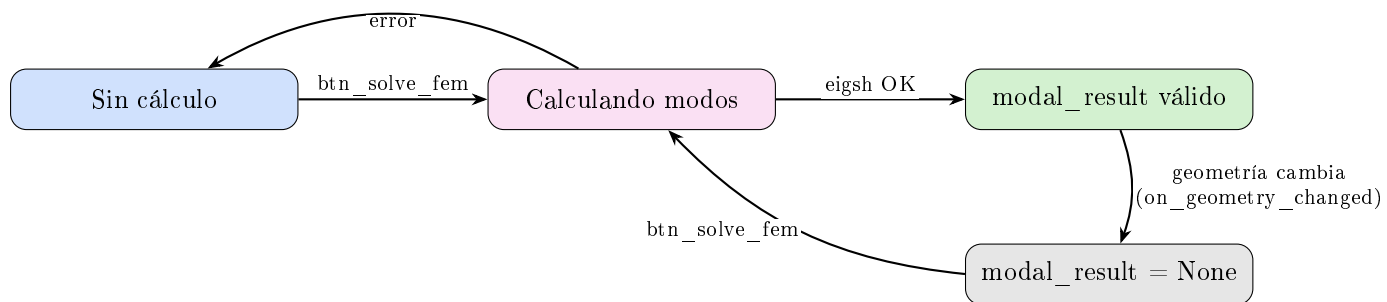


Figura 9: Ciclo de vida del objeto ModalSolution. Cualquier cambio de geometría invalida el resultado.

11. Consideraciones de implementación

11.1. Orden de renderizado en OpenGL

Los ítems del `GLViewWidget` se renderizan en el orden en que fueron agregados. Las esferas de fuentes (`GLScatterPlotItem` con `pxMode=True`) deben agregarse *después* del mesh del recinto para garantizar visibilidad. Por eso `_on_params()` llama a `_refresh_sources_list()` tras `update_geometry()`.

11.2. Proyección manual de la cámara

pyqtgraph 0.14 cambió la signatura de `projectionMatrix()`. La aplicación construye manualmente las matrices `view` y `projection` con `QMatrix4x4` para el cálculo de etiquetas de aristas y picking 2D, sin depender de la API interna de pyqtgraph.

11.3. Debounce del campo acústico

El arrastre de fuentes emite `sourceMoveRequested` en cada evento de mouse (potencialmente 60 veces por segundo). Un `QTimer` monoshot de 350 ms actúa como debounce: el campo se recalcula solo cuando el usuario detiene el arrastre, evitando saturar la CPU con cálculos FEM intermedios.

11.4. Interpolación baricéntrica en mallas tet

El `FieldEvaluator` pre-calcula matrices de transformación afín A_e^{-1} para cada tetraedro. La localización de un punto $\mathbf{x}$ se realiza buscando el tetraedro con máximo mínimo de coordenadas baricéntricas $\min(N_0, N_1, N_2, N_3)$, lo que selecciona el elemento más “interno” al punto en caso de múltiples candidatos.